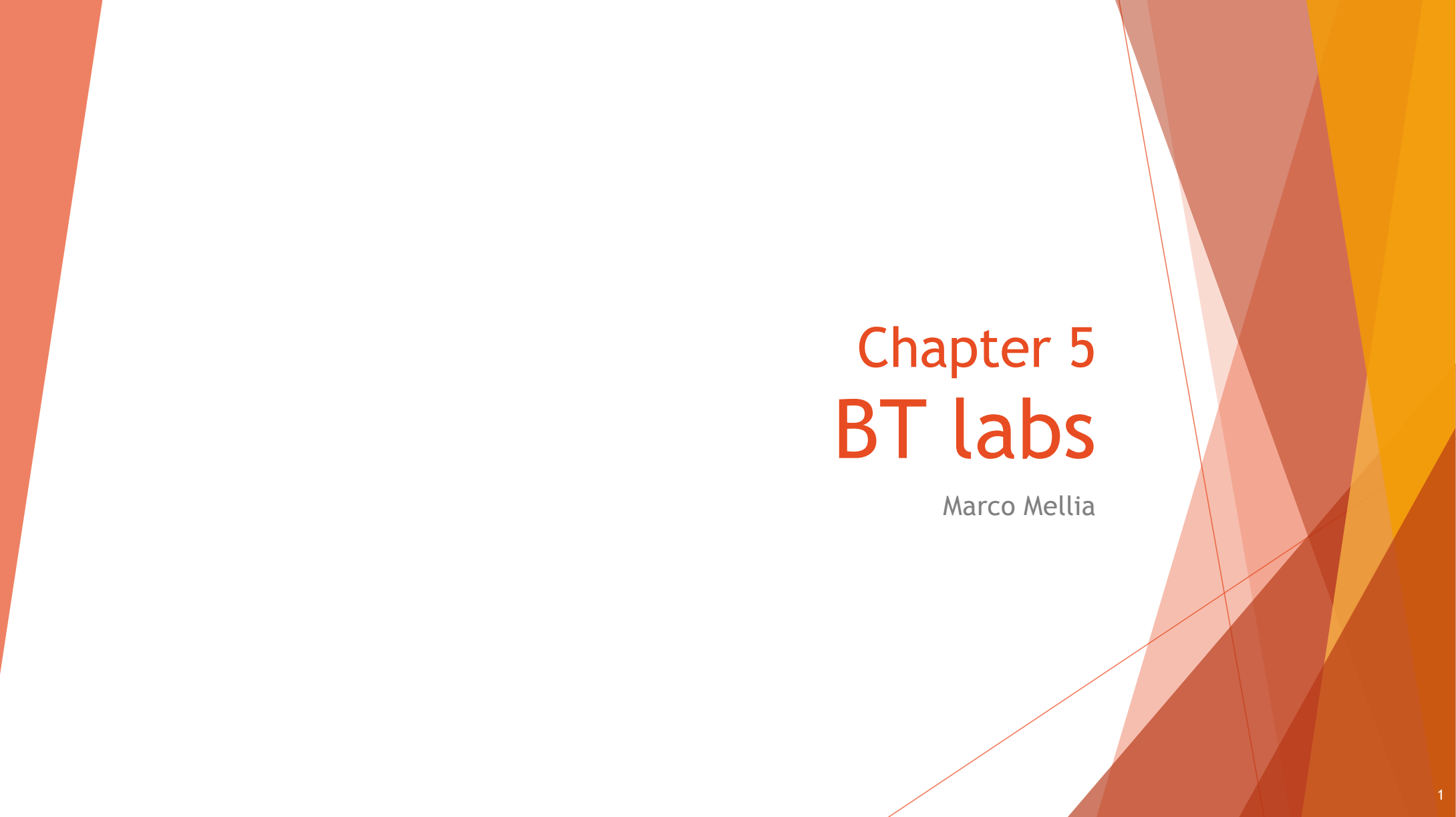




# Chapter 5 BT labs

Marco Mellia

# Bluetooth lab goals

## *Scenario:*

- ▶ You are analysing the Bluetooth communication of an unknown consumer IoT device.
- ▶ Your goal is to observe its behaviour during discovery, pairing, and connection, and assess the security properties of the protocol exchanges using only HCI-level traces.

## Environment constraints

Host Controller Interface

- ▶ Laptop with BT adapter
  - ▶ Linux or MacOS. Even Android or iPhone or Windows
- ▶ A consumer Bluetooth device (e.g., headset, tracker, smart appliance)
- ▶ No access to device internals
  - ▶ Capture via HCI only
- ▶ Only over-the-air interaction via standard pairing

## Learning objectives

- ▶ Understand BT protocol stack at HCI level
- ▶ Observe pairing procedures (Legacy vs Secure Connections)
- ▶ Analyse privacy features (address randomisation, filtering)
- ▶ Use Wireshark filters effectively

# Lab requirements - Linux

- ▶ A Linux with BT interface
  - ▶ Be sure to have the **bluez** stack installed and the Bluetooth service **enabled**
    - ▶ `sudo systemctl start bluetooth.service` [use `enable` to make the change permanent]
    - ▶ This will start some daemons that implement the upper part of the BT software stack
- ```
$ ps ax | grep blue
 1563 ?          Sl      0:01 /usr/bin/python3 /usr/bin/blueman-applet
 1769 ?          Ss      0:00 /usr/libexec/bluetooth/obexd
 2530 ?          Ss      0:00 /usr/libexec/bluetooth/bluetoothd
 2533 ?          Sl      0:03 /usr/bin/python3 /usr/bin/blueman-tray
10109 pts/4    S+      0:00 grep --color=auto blue
```
- ▶ Use Wireshark to analyse packets and filter packets - <https://www.wireshark.org>
    - ▶ You can use the `bluetooth0` interface to capture traffic

# Using the BT linux stack

- ▶ **bluetoothctl**: - interactive Bluetooth control tool
  - ▶ This is an interactive command line interface to interact with the BT stack
  - ▶ There is an online help, accessed via the `help` command

- ▶ Useful commands to manage your device status:

**list**

List available **controllers**

**show** [ctrl]

Controller **information**

**devices** [Paired/Bonded/Trusted/Connected]

List available devices, with an optional property as the filter

**power** <on/off>

Set controller power

**pairable** <on/off>

Set controller pairable mode

**discoverable** <on/off>

Set controller discoverable mode

**advertise** <on/off/type>

Enable/disable advertising with the given type

# Using the BT linux stack

## ► Useful commands:

`scan` <on/off/bredr/le>

`info` [dev/set]

`pair` [dev]

`cancel-pairing` [dev]

`block` [dev]

`unblock` [dev]

`remove` <dev>

`connect` <dev>

`disconnect` [dev]

`Scan` for devices

Device/Set information

`Pair` with device

Cancel pairing with device

Block device

Unblock device

Remove device

Connect device

Disconnect device

# Configuring the BT interface

- ▶ Via bluetoothctl, enter the mgmt menu

```
menu mgmt
```

- ▶ Use the info command to show the status of your device

```
[bluetooth]# info
[bluetooth]# Index list with 1 item
[bluetooth]# hci0:      Primary controller
[bluetooth]#      addr 04:7F:0E:66:3F:8B version 10 manufacturer 2279 class 0x6c0000
[bluetooth]#      supported settings: powered connectable fast-connectable discoverable
bondable link-security ssp br/edr le advertising secure-conn debug-keys privacy static-addr
phy-configuration
[bluetooth]#      current settings: powered bondable br/edr le secure-conn
[bluetooth]#      name kali
[bluetooth]#      short name
```

# Configuration parameters

## ► Useful configuration commands in the menu -> mgmt section

|                                     |                        |
|-------------------------------------|------------------------|
| <code>ssp</code> <on/off>           | Toggle SSP mode        |
| <code>sc</code> <on/off/only>       | Toggle SC support      |
| <code>hs</code> <on/off>            | Toggle HS support      |
| <code>le</code> <on/off>            | Toggle LE support      |
| <code>advertising</code> <on/off>   | Toggle LE advertising  |
| <code>bledr</code> <on/off>         | Toggle BR/EDR support  |
| <code>privacy</code> <on/off> [irk] | Toggle privacy support |

## ► Useful configuration commands in the menu -> scan section

|                                      |                               |
|--------------------------------------|-------------------------------|
| <code>rssi</code> [rss]              | Set/Get RSSI filter           |
| <code>duplicate-data</code> [on/off] | Set/Get duplicate data filter |

► Note: the support for these features may change based on the hardware and driver version

# List of actions

## ▶ Step 1: Get into Bluetoothctl's Interactive Mode

~\$ `bluetoothctl`

- ▶ Notice, how the prompt is changing from ~\$ to [bluetooth]#

## ▶ Now, you can execute any command like help after the # symbol

- ▶ Use `help` to find all existing commands and their one-liner explanations

## ▶ Step 2: Turn on Bluetooth in Linux

[bluetooth]# `power on`

- ▶ You get an output like:

```
[CHG] Controller F8:89:D2:C8:2E:54 Class: 0x006c010c  
Changing power on succeeded
```

- ▶ You can use the `power off` command to turn it off



# List of actions

## ▶ Step 3: Scan for Available Bluetooth Devices

```
[bluetooth]# scan on
```

- ▶ As you turn on your Bluetooth device, `bluetoothctl` will list it in the output
- ▶ After you have found your device, copy its address for future use

## ▶ Question:

- ▶ What changes is you scan only BREDR or LE devices?
- ▶ Try `[bluetooth]# scan le` or `[bluetooth]# scan bredr`

## ▶ If you want your Linux computer to be discoverable by other devices, set it as discoverable

```
[bluetooth]# discoverable on
```

- ▶ This is not necessary

## ▶ Step 4: List devices

```
[bluetooth]# devices
```

- ▶ This will list all discovered devices
- ▶ You can list `Paired`, `Bonded`, and `Trusted` devices, too

# List of actions

## ▶ Step 5: Connect to a Bluetooth Device

- ▶ Select one of the devices by finding its MAC address
  - ▶ Eventually make it discoverable (e.g. your mouse)
- ▶ Start the pairing on your Linux computer using

```
[bluetooth]# pair 90:78:B2:C7:8F:A8
```

- ▶ Depending on the type of device, you might have to accept the connection with the PIN eventually

## ▶ You can then connect with the same device

```
[bluetooth]# connect 90:78:B2:C7:8F:A8
```

## ▶ Step 6: List device capabilities

```
bluetooth]# info 90:78:B2:C7:8F:A8
```

# Wireshark

- ▶ Capture packets with Wireshark
  - ▶ Note: Wireshark captures the messages on the HCI interface - not on the physical interface
  - ▶ There is no equivalent of “monitor mode” with standard BT interfaces
- ▶ Use **filters** to select the packets
  - ▶ By address: `bluetooth.addr == 20:f4:78:5c:96:d9`
  - ▶ By protocol: `_ws.col.protocol == "SMP"`
  - ▶ Removing advertising reports: `!(bthci_evt.code == 0x3e)`
  - ▶ ... use the **right-click** -> apply as a filter -> options

## LAB TODO

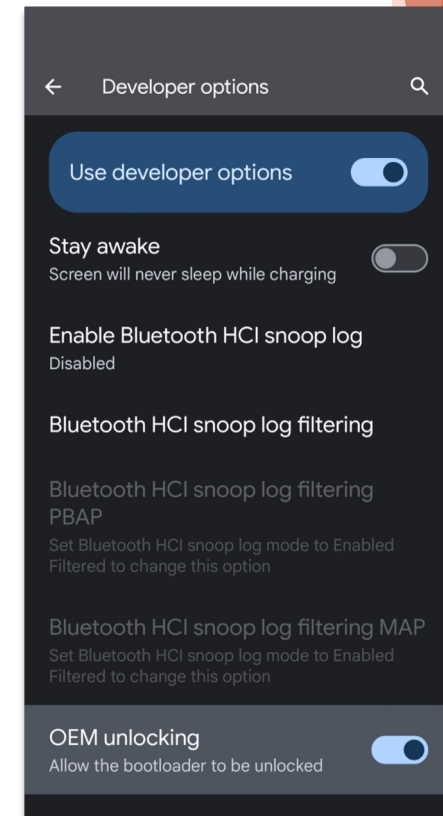
- ▶ Choose one device to explore the BT capabilities and protocol
- ▶ Use Wireshark to capture packets
- ▶ Use the client to perform
  - ▶ Scan operations
  - ▶ Pairing operations
  - ▶ Connecting
- ▶ Observe on Wireshark the
  - ▶ Pairing procedures
  - ▶ Capability listing
  - ▶ ...
- ▶ Verify that messages follow the expected format
  - ▶ For privacy, the system may not let you/Wireshark see some messages
  - ▶ Can you see the Secure Connections messages? Why?

# Lab requirements - MacOS

- ▶ A Macbook with BT interface
- ▶ Install **Xcode**
  - ▶ Visit the App Store or this [link](#) to install Xcode on your Mac
- ▶ Install the **packetLogger** tool
  - ▶ Download the **Additional Tools for Xcode** via this [link](#)
    - ▶ When downloaded successfully, open the Additional\_Tools\_for\_Xcode\_XX.dmg and access the Hardware folder. The packetLogger is inside the Hardware folder
  - ▶ Download the **logging profile** from <https://developer.apple.com/feedback-assistant/profiles-and-logs/> and install it
  - ▶ Activate it in the Preferences -> General -> Device Management
- ▶ It can also be used to capture BT packets from an iOS device remotely
  - ▶ See <https://www.bluetooth.com/blog/a-new-way-to-debug-iosbluetooth-applications/> for details
- ▶ Use Wireshark to analyse packets and filter packets - <https://www.wireshark.org>

# Lab requirement - Android

- ▶ On Android devices, it is possible to capture Bluetooth traffic as follows:
  - ▶ Go to Settings
  - ▶ Enable Developer mode (tap 7 times on the “Build number” in the “Software information” menu)
  - ▶ Go into developer options
  - ▶ Enable the option Enable Bluetooth HCI snoop log
  - ▶ Enable the Bluetooth option and Connect to the device
- ▶ Transfer the file to the PC
  - ▶ Make sure the device is connected to the PC
  - ▶ The files might be shown in a PC's file browser in 'Internal Storage'
  - ▶ If not, use the Android Debug Bridge: `adb pull /sdcard`
- ▶ The files of interest are **btsnoop\_hci.log** and all files with the extension **.cfa**
  - ▶ These are binary files, which can be opened with Wireshark
- ▶ Remember to disable the logging at the end!
  - ▶ Turn off the Bluetooth on the device
  - ▶ Disable the option Enable Bluetooth HCI snoop log



# On windows [untested]

- ▶ You can get the Microsoft Bluetooth Test Platform software package
  - ▶ Download and install it from <https://learn.microsoft.com/en-us/windows-hardware/drivers/bluetooth/testing-btp-setup-package>
- ▶ Use the Bluetooth Virtual Sniffer (btvs.exe) to capture HCI traces
  - <https://learn.microsoft.com/en-us/windows-hardware/drivers/bluetooth/testing-btp-tools-btvs>

## Wireshark operation

- ▶ Usage for Wireshark on the same machine
  - ▶ Run btvs.exe using the command prompt\PowerShell console:  
`btvs.exe -Mode Wireshark`
    - ▶ If Wireshark is installed, Wireshark opens automatically
- ▶ Otherwise, manually start Wireshark and provide the default TCP pipe as the interface:  
`wireshark -k -i TCP@127.0.0.1:24352`

# LAB 1 - MAC addresses anonymisation

## Objective

- ▶ Understand BLE address **randomisation** (RPA)
- ▶ Verify that:
  - ▶ A **passive observer** sees **rotating RPAs**
  - ▶ A **paired device** resolves identity via **IRK**
- ▶ Attempt **linkability attacks** via **payload correlation**

## Roles / Topology

- ▶ **Alice Device** (target)
  - ▶ BLE device advertising (e.g. phone, dev board)
- ▶ **Alice Laptop** (trusted)
  - ▶ **Paired with Alice device**
  - ▶ Can resolve **RPA → identity**
- ▶ **Trudy Laptop** (attacker)
  - ▶ **Passive sniffer** only
  - ▶ No pairing, **no IRK**



# LAB 1 - MAC Address anonymisation

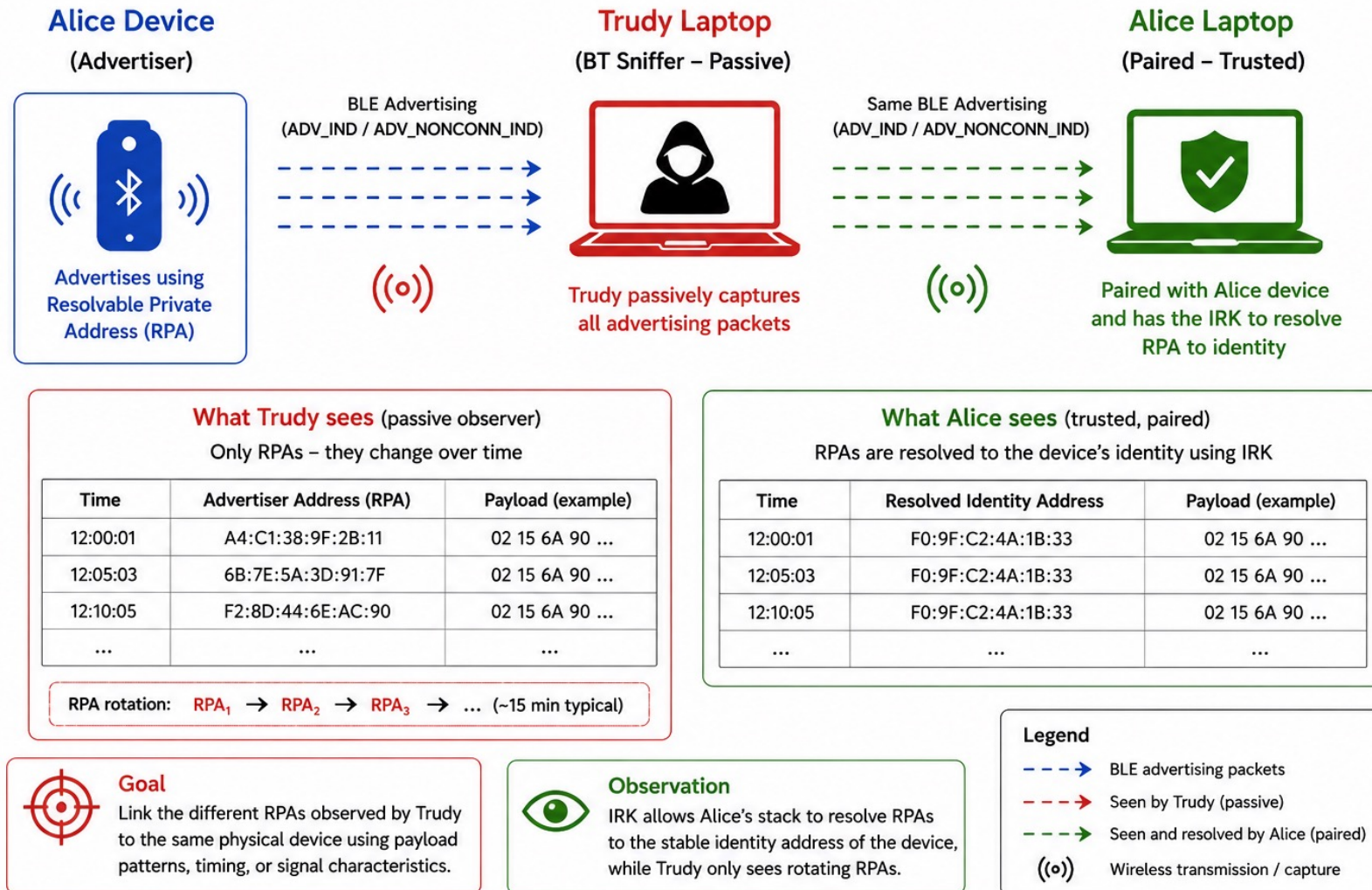
## Observe RPA

### ▶ Procedure

- ▶ Alice device advertises normally
- ▶ Alice runs PacketLogger/MacOS or Wireshark/Linux
  - ▶ Stack resolves RPA → Identity address (via IRK)
  - ▶ Alice sees the resolved address
- ▶ Trudy runs PacketLogger/MacOS or Wireshark/Linux
  - ▶ Trudy seen the RPA

### ▶ Can payload-level fingerprints deanonymize devices?

# Attack Setup – MAC Address Anonymisation in BLE



# LAB 1 - Lab Deliverables: RPA Analysis

- ▶ **Goal: Demonstrate BLE MAC address anonymisation and its limitations using two viewpoints**
  - ▶ Alice (paired, trusted) → resolves identity
  - ▶ Trudy (passive observer) → sees only RPAs
- ▶ **Traces to collect**
  - ▶ **Alice trace (trusted)**
    - ▶ Capture while connecting to the device (after pairing)
    - ▶ Advertisement messages show:
      - ▶ Resolved identity address
      - ▶ Corresponding BLE traffic (advertising / connection)
  - ▶ **Trudy trace (passive)**
    - ▶ Capture the same interaction without pairing
    - ▶ Show:
      - ▶ Rotating Resolvable Private Addresses (RPAs)
      - ▶ Same device advertising activity as seen by Alice

# LAB 1 - Lab Deliverables: RPA Analysis

## ▶ Required evidence

- ▶ Identify the **same device** in both traces:
  - ▶ Alice: stable identity address
  - ▶ Trudy: changing RPAs
- ▶ **Show at least one RPA rotation event**
- ▶ Provide **Wireshark filters** used to isolate the traffic

## ▶ Analysis

- ▶ Explain:
  - ▶ Why Alice can resolve the identity (IRK)
  - ▶ Why Trudy cannot
- ▶ **Attempt linkability:**
  - ▶ Use payload patterns, timing, or RSSI
  - ▶ Can you track the device across RPA changes?

## ▶ Prepare one .zip archive

- ▶ Archive name: **Sxxxxxx\_BTLAB.zip**
- ▶ Inside two filtered traces and a **Setup.txt**
  - ▶ Sxxxxxx\_DeviceName\_Alice.pcap
  - ▶ Sxxxxxx\_DeviceName\_Trudy.pcap
  - ▶ **Setup.txt** including:
    - ▶ Device type and vendor
    - ▶ OS and tools used
    - ▶ BT adapter
    - ▶ List of RPA identified in Trudy's trace
  - ▶ **One plot** showing the timeseries of different MAC addresses in Trudy's trace

## ▶ Deliverable:

- ▶ **Upload your .zip file in this SHARE**

<https://www.dropbox.com/request/drl4jkiehro4e7hs05js>

# LAB 1 - MAC Address anonymisation

## ▶ Linkability attack

### ▶ From Alice's trace:

- ▶ Identify all packets sent by Alice's Device using its public address

```
bthci_evt.bd_addr == <PMAC>
```

- ▶ Identify patterns/payloads in Alice Device that are specific to such a device, e.g.,

```
btcommon.eir_ad.entry.data == <payload>
```

```
07:19:01:19:20:35:aa:b7:39:00:04:e9:20:da:8e:28:0f:06:24:07:a4:1b:67:ec:02:e7:dd
```

### ▶ From Trudy's trace:

- ▶ Identify packets carrying that pattern

```
btcommon.eir_ad.entry.data ==
```

```
07:19:01:19:20:35:aa:b7:39:00:04:e9:20:da:8e:28:0f:06:24:07:a4:1b:67:ec:02:e7:dd
```

- ▶ Track them across address changes => filter by this RPA

```
bthci_evt.bd_addr == <RPA1>
```

### ▶ Check and Repeat

## ▶ Can payload-level fingerprints deanonymize devices?

# LAB 1 - MAC Address anonymisation

## ► Linkability attack

### ► From Alice's trace:

- Identify all packets sent by Alice's Device using its public address

```
bthci_evt.bd_addr == <PMAC>
```

- Identify patterns/payloads in Alice Device that are specific to such a device

```
btcommon.eir_ad.entry.data == <payload>
```

```
07:19:01:19:20:35:aa:b7:39:00:04:e9:20:da:8e:28:0f:06:24
```

### ► From Trudy's trace:

- Identify packets carrying that pattern

```
btcommon.eir_ad.entry.data ==
```

```
07:19:01:19:20:35:aa:b7:39:00:04:e9:20:da:8e:28:0f:06:24
```

- Track them across address changes => filter by this RPA

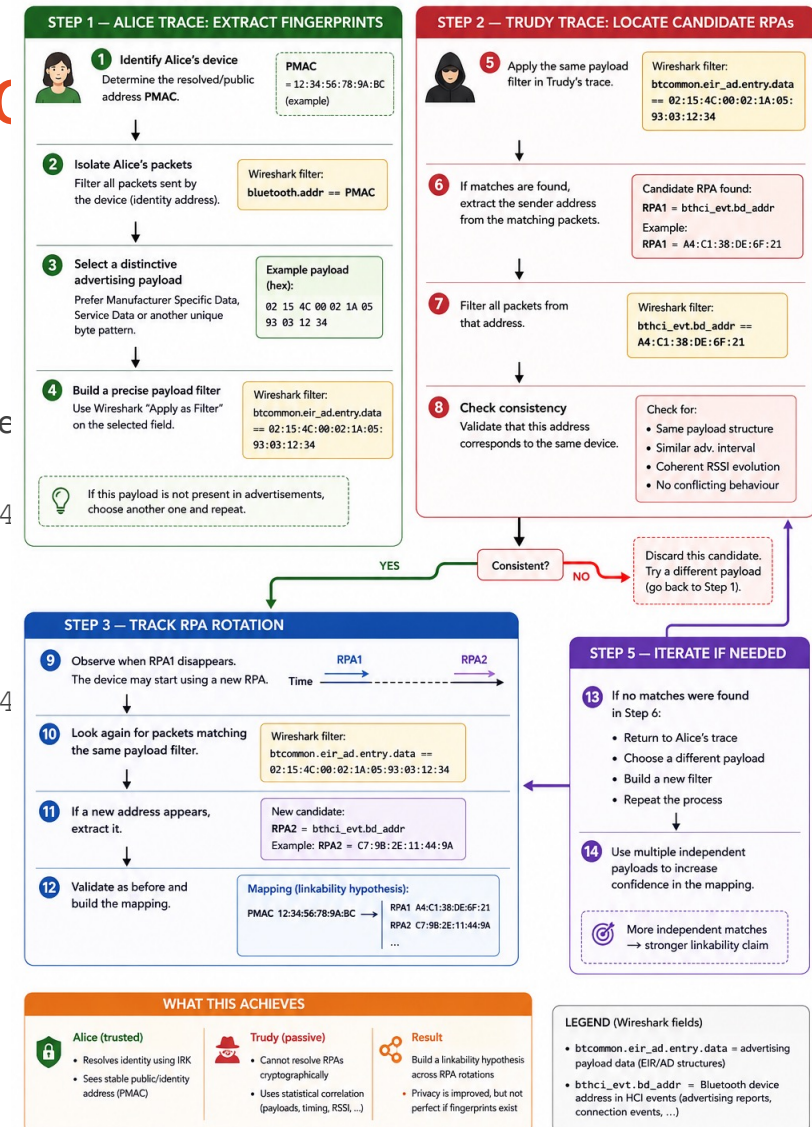
```
bthci_evt.bd_addr == <RPA1>
```

### ► Check and Repeat

## ► Can payload-level fingerprints deanonymize devices?

### Algorithm: Identifying Alice's RPAs in Trudy's Trace

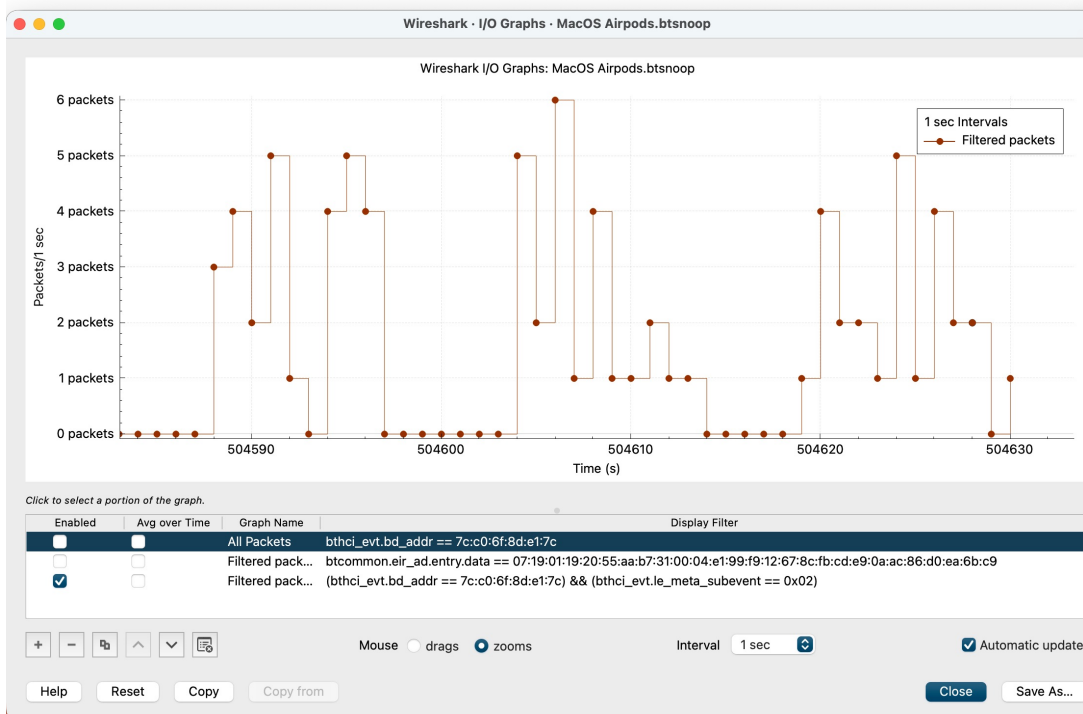
Alice is paired (identity resolved). Trudy is a passive observer (sees only RPAs).





# Lab 1 - I/O graph plots

Alice's trace - 1 PMAC



Trudy's trace - 3 RPAs

